

1) What is File Handling ?

- File Handling is the storing of data in a file using a program.
- File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program.

2) Why use File Handling ?

- Store large data permanently.
- Avoid re-entering data every time program runs.
- Easy to share or reuse data between programs.

3) Operations on Files :-

The main operations are:

1. Create - Make a new file
2. Open - Open an existing file.
3. Read - Get data from file.
4. Write - Save data into file
5. Append - Add data at the end of a file (without erasing old content).
6. Close - Release the file after use
7. Delete (optional, using remove() function).

4) File Opening Modes :-

When you open a file using fopen(), you must choose a mode:

Mode	Meaning
"r"	Open for reading (file must exist).
"w"	Open for writing (erases file if it exists, creates new file).
"a"	Open for appending (data is added at end).
"r+"	Open for reading and writing (file must exist)
"w+"	Open for reading and writing (erases file if exists, new file created).
"a+"	Open for reading and appending (file created if not exists).
"rb", "wb"	Same as above but for binary files (images, executables).

5) Important Functions :-

Used from <stdio.h> library:

1. FILE *fopen(const char *filename, const char *mode); - Opens file.
2. int fclose(FILE *stream); - Closes file.
3. int fprintf(FILE *stream, const char *format); - Writes formatted text.

4. int fscanf(FILE *stream, const char *format);- Reads formatted text
5. int fgetc(FILE *stream); - Reads one character.
6. int fputc(int char, FILE *stream); - Writes one character
7. char *fgets(char *str, int n, FILE "stream); - Reads string.
8. int fputs(const char *str, FILE *stream); - Writes string.
9. int feof(FILE *stream); - Checks End of File.
10. int removef(const char *filename); -Deletes file.

Example:

```
#include <stdio.h>

int main( )
{
    FILE *fptr;
    char str[50];

    // Writing to a file
    fptr= fopen("test.txt", "w");
    fprintf(fptr, "Hello, this is file handling!");
    fclose(fptr);

    // Reading from a file
    fptr = fopen("test.txt", "r");
    fgets(str, 50, fptr);
    printf("File content: %s\n", str);
    fclose(fptr);
    return 0;
}
```

6) Function in File Handling :-

File handling functions in C are used to create, open, read, write, and close files for permanent data storage. These functions are defined in the stdio.h header file.

Function	Operation
fopen()	Creates a new file / opens an existing file
fclose()	Closes a file which has been opened for use
getc()	Reads a character from the file
putc()	Writes a character to the file
fprintf()	Write data values to a file
fscanf()	Reads a set of data values from a file
getw()	Reads an integer from the file
putw()	Writes an integer to a file
fseek()	Sets the position to the desired point in the file
ftell()	Gives the current position in the file
rewind()	Sets the position to the beginning of the file

1. fopen()

Theory : Opens a file in the specified mode (read, write, append, etc.). Returns a pointer to FILE.

Syntax:

```
FILE *fopen(const char *filename, const char *mode);
```

Example:

```
FILE *fp;
```

```
fp = fopen("data.txt", "r"); // open file for reading
```

2. fclose()

Theory : Closes an open file. It flushes all buffers and releases resources.

Syntax:

```
int fclose(FILE *stream);
```

Example:

```
fclose(fp); // Close the file
```

3. fputs()

Theory : Writes a string to a file.

Syntax:

```
int fputs(const char *str, FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "w");  
fputs("Hello File Handling!", fp);  
fclose(fp);
```

4. fgets()

Theory : Reads a line (string) from a file.

Syntax:

```
char *fgets(char *str, int n, FILE *stream);
```

Example:

```
char buffer[100];  
FILE *fp = fopen("data.txt", "r");  
fgets(buffer, 100, fp);  
printf("%s", buffer);  
fclose(fp);
```

5. fprintf()

Theory: Writes formatted output to a file (like printf but for files).

Syntax:

```
int fprintf(FILE *stream, const char *format, ...);
```

Example:

```
FILE *fp = fopen("student.txt", "w");  
int roll = 1;  
char name[] = "Sagar";  
fprintf(fp, "Roll No: %d\nName: %s\n", roll, name);
```

```
fclose(fp);
```

6. fscanf()

Theory : Reads formatted input from a file (like scanf but for files).

Syntax:

```
int fscanf(FILE *stream, const char *format, ...);
```

Example:

```
int roll;  
char name[20];  
FILE *fp = fopen("student.txt", "r");  
fscanf(fp, "Roll No: %d\nName: %s", &roll, name);  
printf("Roll: %d, Name: %s", roll, name);  
fclose(fp);
```

7. fgetc()

Theory : Reads a single character from a file.

Syntax:

```
int fgetc(FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "r");  
char ch = fgetc(fp);  
printf("First character: %c", ch);  
fclose(fp);
```

8. fputc()

Theory : Writes a single character to a file.

Syntax:

```
int fputc(int char, FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "w");  
fputc('A', fp);  
fclose(fp);
```

9. feof()

Theory : Checks if the end of the file is reached.

Syntax:

```
int feof(FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "r");  
while(!feof(fp)) {  
    char ch = fgetc(fp);  
    printf("%c", ch);  
}  
fclose(fp);
```

10. fseek() and ftell()

fseek(): Move file pointer

```
int fseek(FILE *stream, long int offset, int whence);
```

ftell(): Get current position of file pointer

```
long int ftell(FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "r");  
fseek(fp, 5, SEEK_SET); // Move to 5th byte  
printf("Position: %ld", ftell(fp));  
fclose(fp);
```

11.rewind()

Theory : Sets the file pointer to the beginning of the file.

Syntax:

```
void rewind(FILE *stream);
```

Example:

```
FILE *fp = fopen("data.txt", "r");
```

```
fseek(fp, 10, SEEK_SET);
```

```
rewind(fp); // Go back to start
```